

Mtech Software Engineering

API-Based Products (S2-25_SEZG504)

Name :Salandri Shirisha: 2025TM93131

Github: <https://github.com/2025tm93131-shiri/record-label>

1. Provide the Open API 3.1.1 specification for designing an API for a record label. Let's assume that the record label has a database of artists with the following information:

- Artist name
- Artist genre
- Number of albums published under the label
- Artist username

Overview

This API is designed to manage artists in a record label system. It supports retrieving artist details, adding new artists, and fetching specific artist information. The API is secured using Basic Authentication so that only authorized users can access it.

The API is designed with proper REST principles:

- Secure access using authentication
- Efficient data retrieval using pagination
- Clear request/response structure using schemas

openapi: 3.1.1

info:

title: Record Label API

version: 1.0.0

description: API Assignment for managing artists in a record label

servers:

- url: http://localhost:3000

components:

securitySchemes:

basicAuth:
 type: http
 scheme: basic

schemas:

Artist:
 type: object
 properties:
 id:
 type: integer
 example: 1
 name:
 type: string
 example: Drake
 genre:
 type: string
 example: Hip-Hop
 albumCount:
 type: integer
 example: 8
 username:
 type: string
 example: drake
 required: [id, name, genre, albumCount, username]

ArtistInput:
 type: object
 properties:
 name:
 type: string
 genre:
 type: string
 albumCount:
 type: integer
 username:
 type: string
 required: [name, genre, albumCount, username]

Error:

type: object

properties:

message:

type: string

example: Unauthorized

paths:

/artists:

get:

tags: [Artists]

operationId: getArtists

summary: Get list of artists

security:

- basicAuth: []

parameters:

- name: offset

in: query

schema:

type: integer

default: 0

- name: limit

in: query

schema:

type: integer

default: 10

responses:

"200":

description: List of artists

content:

application/json:

schema:

type: array

items:

\$ref: '#/components/schemas/Artist'

"400":
 description: Invalid query params
"401":
 description: Unauthorized
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/Error'

post:
 tags: [Artists]
 operationId: createArtist
 summary: Add a new artist
 security:
 - basicAuth: []
 requestBody:
 required: true
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/ArtistInput'
 responses:
 "201":
 description: Artist created
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/Artist'
 "400":
 description: Bad request
 "401":
 description: Unauthorized

/artists/{name}:
 get:
 tags: [Artists]
 operationId: getArtistByName

summary: Get artist by name

security:

- basicAuth: []

parameters:

- name: name

in: path

required: true

schema:

type: string

responses:

"200":

description: Artist found

content:

application/json:

schema:

\$ref: '#/components/schemas/Artist'

"404":

description: Artist not found

"401":

description: Unauthorized

HTTP <http://localhost:3000/artists> Save Share

GET <http://localhost:3000/artists?offset=0&limit=3> Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Body 200 OK • 6 ms • 482 B

{ } JSON Preview Visualize

```
1 {
2   {
3     "id": 1,
4     "name": "salandri shirisha",
5     "genre": "Hip-Hop",
6     "albumCount": 8,
7     "username": "shiri"
8   },
9   {
10    "id": 2,
11    "name": "Drake",
12    "genre": "Hip-Hop",
13    "albumCount": 2,
14    "username": "drake"
15  },
16  {
17    "id": 3,
18    "name": "Taylor Swift",
19    "genre": "Pop",
20    "albumCount": 12,
21    "username": "taylor"
22  }
23 }
```

Navigator: SCHEMAS

- record_label
 - Tables
 - artists
 - Views
 - Stored Procedures
 - Functions
- sys

Query 1 new_schema - Schema record_label.artists artists - Table artists

Limit to 1000 rows

1 • SELECT * FROM record_label.artists;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: FA

	id	name	genre	albumCount	username
▶	1	salandri shirisha	Hip-Hop	8	shiri
	2	Drake	Hip-Hop	2	drake
	3	Taylor Swift	Pop	12	taylor
	4	Arijit Singh	Bollywood	10	arijit
*	NULL	NULL	NULL	NULL	NULL

Administration Schemas

2. Perform rate limiting and request size limiting with KONG API Gateway.

Overview

Kong API Gateway is used to manage and control traffic to the API. It helps in improving performance, security, and reliability.

Setup

- Kong was deployed using Docker
- The Node.js API was connected as a service in Kong
- Routes were configured to expose the /artists endpoint

Rate Limiting

Rate limiting restricts the number of API requests a client can make within a given time.

Configuration:

- Limit: 5 requests per minute

Testing:

- Multiple requests were sent repeatedly
- After exceeding the limit, the API returned:

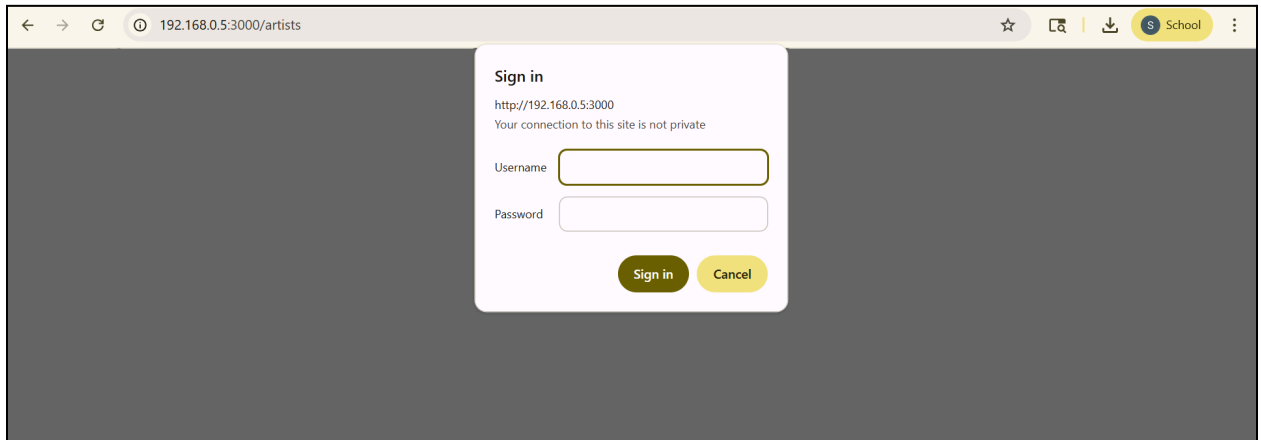
429 Too Many Requests

Observation:

This prevents abuse and ensures fair usage of the API.

Request Size Limiting

This restricts the size of the request payload sent by the client.





⚠ Not secure

192.168.0.5:3000/artists

Pretty-print ☒

```
[
  {
    "id": 1,
    "name": "salandri shirisha",
    "genre": "Hip-Hop",
    "albumCount": 8,
    "username": "shiri"
  },
  {
    "id": 2,
    "name": "Drake",
    "genre": "Hip-Hop",
    "albumCount": 2,
    "username": "drake"
  },
  {
    "id": 3,
    "name": "Taylor Swift",
    "genre": "Pop",
    "albumCount": 12,
    "username": "taylor"
  },
  {
    "id": 4,
    "name": "Arijit Singh",
    "genre": "Bollywood",
    "albumCount": 10,
    "username": "arijit"
  }
]
```

Not secure192.168.0.5:3000/artists

Pretty-print

[
 {
 "id": 1,
 "name": "salandri shirisha",
 "genre": "Hip-Hop",
 "albumCount": 8,
 "username": "shiri"
 },
 {
 "id": 2,
 "name": "Drake",
 "genre": "Hip-Hop",
 "albumCount": 2,
 "username": "drake"
 },
 {
 "id": 3,
 "name": "Taylor Swift",
 "genre": "Pop",
 "albumCount": 12,
 "username": "taylor"
 },
 {
 "id": 4,
 "name": "Arijit Singh",
 "genre": "Bollywood",
 "albumCount": 10,
 "username": "arijit"
 }
]

docker desktop

PERSONAL

Search

Ctrl+K

1

Sign in

Containers

Images

Volumes

Kubernetes

Builds

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

Give feedback

Container CPU usage 3.80% / 800% (8 CPUs available)

Container memory usage 752.28MB / 7.48GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last star	Actions
☐	kong-db	b3805a7e07f8	postgres:1	5432:5432	0.53%	2 hours a	
☐	kong	1ccf4914dd95	kong	8000:8000 Show all ports (2)	2.19%	2 hours a	

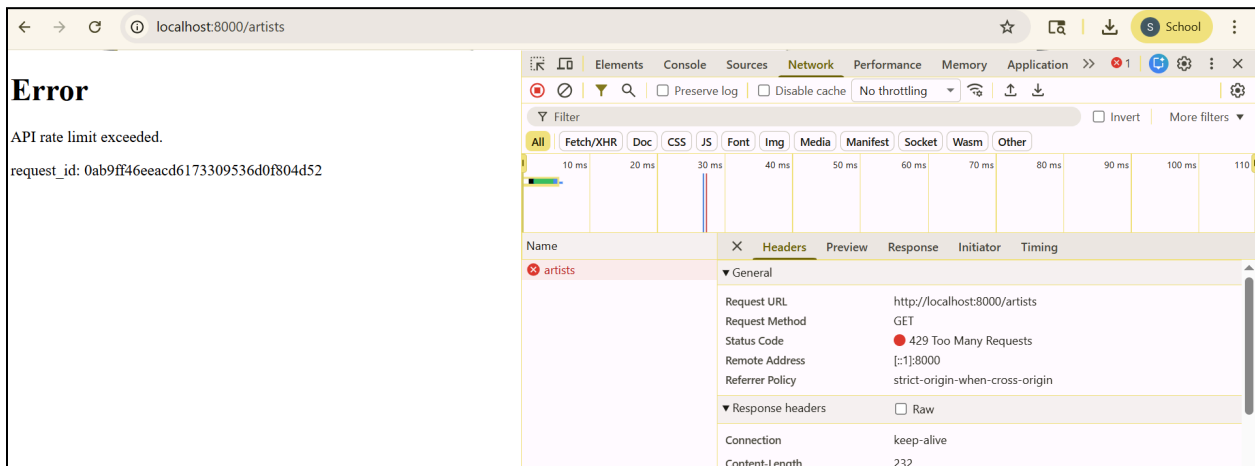
Walkthroughs

```
7 executed
atabase is up-to-date

.\Users\DELL>docker run -d --name kong --link kong-db:kong-db -e KONG_DATABASE=postgres -e KONG_PG_HOST=kong-db -e KONG_PG_PASSWORD=kong -p 8000:8000 -p 8001:8001 kong
e00e0c7bd3167c71c84a838458bab4f497c845b93f262b0e9e5893401bce107

.\Users\DELL>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
a00e0c7bd31   kong          "/docker-entrypoint..." 20 seconds ago Up 19 seconds (healthy) 0.0.0.0:8000-8001->8000-8001/tcp, [::]:8000-8001->8000-8001/tcp   kong
3805a7e07f8   postgres:13   "docker-entrypoint.s..." About a minute ago Up About a minute 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   kong-
b

.\Users\DELL>
```



3. Build a Simple API Demonstrating Different Paradigms

Goal: Implement REST, RPC, and GraphQL for the same domain.

Task: Create a “Book Info Service” using:

- REST (/books, /books/{id})
- RPC (/getBook, /createBook)
- GraphQL (query { book(id:1) { title author } })

Overview

A “Book Info Service” was implemented using three different API paradigms. All three return the same data but differ in design and usage.

Data

Each approach returns data like:

```
{  
  "id": 1,  
  "title": "Atomic Habits",  
  "author": "James Clear"  
}
```

1. REST API

Endpoints:

- GET /books – Get all books
- GET /books/{id} – Get book by ID

Characteristics:

- Resource-based
- Uses HTTP methods like GET
- Simple and widely used

2. RPC API

Endpoints:

- POST /getBook
- POST /createBook

Characteristics:

- Action-based (function-style calls)
- Each endpoint represents an operation
- Less flexible compared to REST

3. GraphQL API

Endpoint:

- /graphql

Query Example:

```
{  
  book(id: 1) {  
    title  
    author  
  }  
}
```

Characteristics:

- Query-based
- Client specifies exactly what data is needed
- Single endpoint for all operations

Comparison of Paradigms

Feature	REST	RPC	GraphQL
Style	Resource-based	Function-based	Query-based
Endpoints	Multiple	Multiple	Single
Flexibility	Medium	Low	High
Data control	Fixed	Fixed	Client decides
Complexity	Low	Low	Moderate

Summary

- REST is simple and widely used
- RPC is straightforward but less flexible
- GraphQL provides more control to clients

Overview GET http://localhost:3000/api/books NO environment

HTTP http://localhost:3000/api/books Save Share

GET http://localhost:3000/api/books Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

Body 200 OK • 7 ms • 406 B

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "title": "Surrounded by Idiots",
5     "author": " Thomas Erikso "
6   },
7   {
8     "id": 2,
9     "title": "Lighter",
10    "author": "yung pueblor"
11  },
12  {
13    "id": 3,
14    "title": "Ikigai",
15    "author": "Hector Garcia"
16  }
17 ]
```

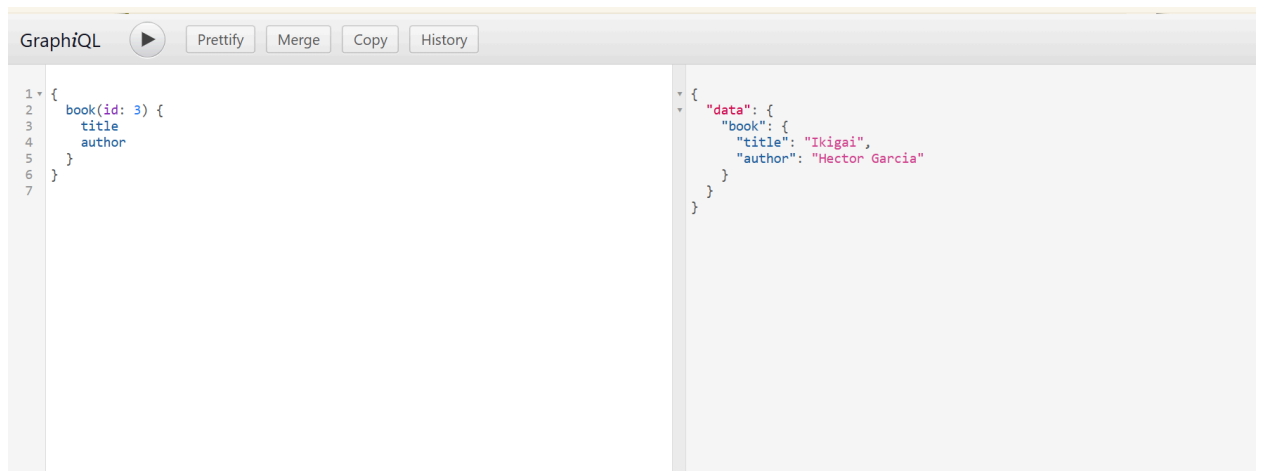
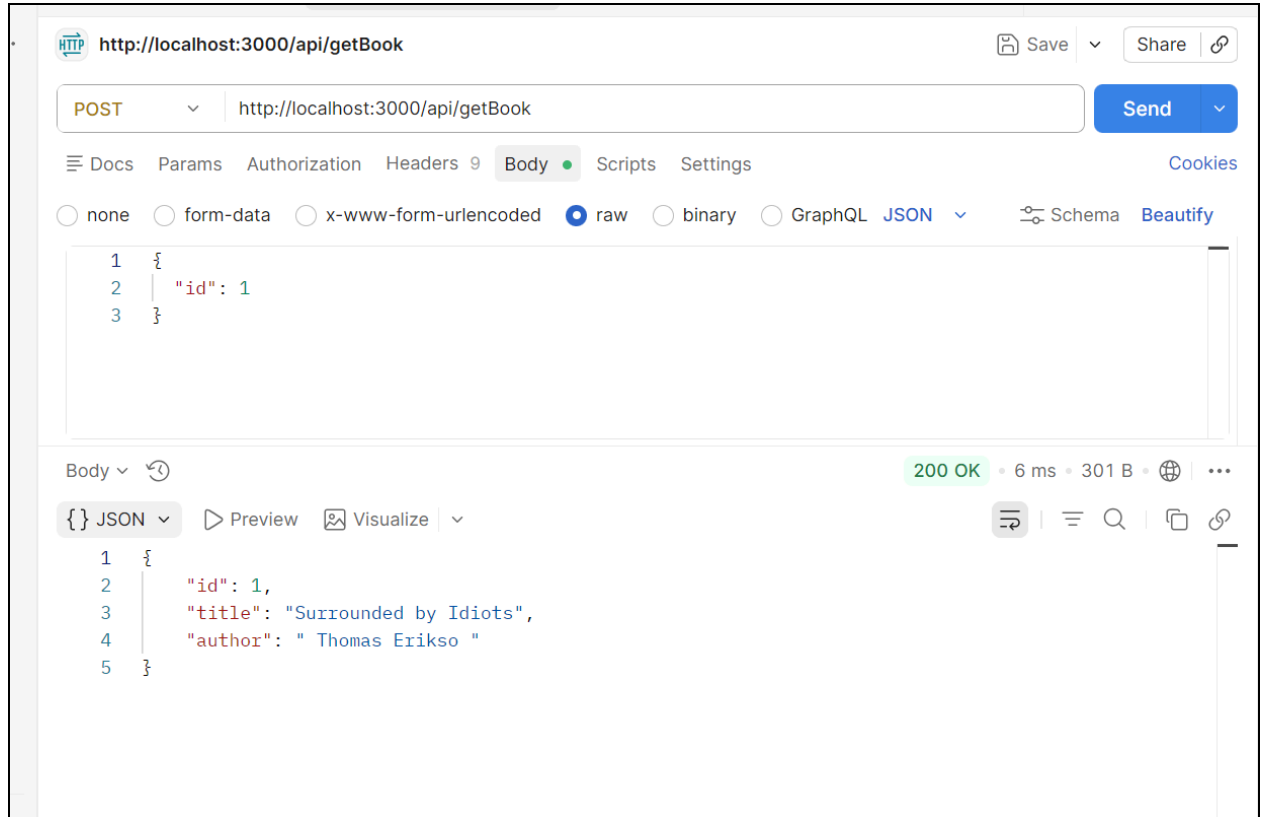
GET http://localhost:3000/api/books/1 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK • 7 ms • 301 B

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "title": "Surrounded by Idiots",
4   "author": " Thomas Erikso "
5 }
```



2025tm93131-shiri / record-label

Q Type to search

CodeIssuesPull requestsActionsProjectsWikiSecurity and qualityInsightsSettings

record-labelPublic

PinWatch 0Fork 0Star 0

main1 Branch0 Tags

Go to fileTAdd fileCode

2025tm93131-shiri add Api for same domain check051385c · now2 Commits

.gitignore	Api initial setup	3 hours ago
README.md	add Api for same domain check	now
api.js	add Api for same domain check	now
db.js	Api initial setup	3 hours ago
package-lock.json	add Api for same domain check	now
package.json	add Api for same domain check	now
server.js	add Api for same domain check	now

README

API Based Product Assignment

1. Provide the Open API 3.1.1 specification for designing an API for a record label. Let's assume that the record label has a database of artists with the following information:

Artist name Artist genre Number of albums published under the label Artist username

2. Create API Gateway (BaaS - Cloud Functions)

About

No description, website, or topics provided.

ReadmeActivity0 stars0 watching0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 2

salandrishirisha

2025tm93131-shiri

Languages

Thank you!